

Connecting X-Heep to External Simulators Through Custom Instructions

1. Context

Modern System-on-Chip (SoC) platforms increasingly rely on custom instructions and tightly coupled accelerators to meet performance and energy efficiency requirements. While SoCs such as X-HEEP provide support for extensibility, integrating new hardware functionality, particularly when introducing custom instruction interfaces like XIF (eXtension InterFace), remains complex and time-consuming. Previous work proposed a software-based co-simulation framework that integrates high-level simulators into an SoC by generating the necessary interface code automatically.

2. Motivation

The original co-simulation framework focuses on peripheral-level integration and does not natively support accelerators tightly coupled to the processor core. As a result, it is difficult to accurately capture instruction-level behavior and processor-accelerator interactions. Furthermore, prototyping XIF-based extensions in X-HEEP typically requires manual hardware modifications, limiting fast experimentation. This work addresses these challenges by extending the framework to support XIF custom instructions, enabling accurate instruction-level modeling while preserving the framework's flexibility and ease of use.

3. Implementation and Features

3.1 Socsim Generator

Socsim Generator is a co-simulation framework designed to automate the integration of high-level simulators with X-Heep. Using a declarative JSON configuration, the framework generates both the communication logic required on the X-Heep side and the base code for the corresponding high-level simulator. This approach removes the need for manual hardware modifications or low-level communication code, enabling fast prototyping and evaluation of custom components.

The framework supports co-simulation through two RISC-V protocols: the Open Bus Interface (OBI) for peripheral-level simulation, and the eXtended InterFace (XIF) for coprocessor integration. While OBI peripherals model external hardware devices, XIF coprocessors allow custom instructions to be executed within the processor pipeline.

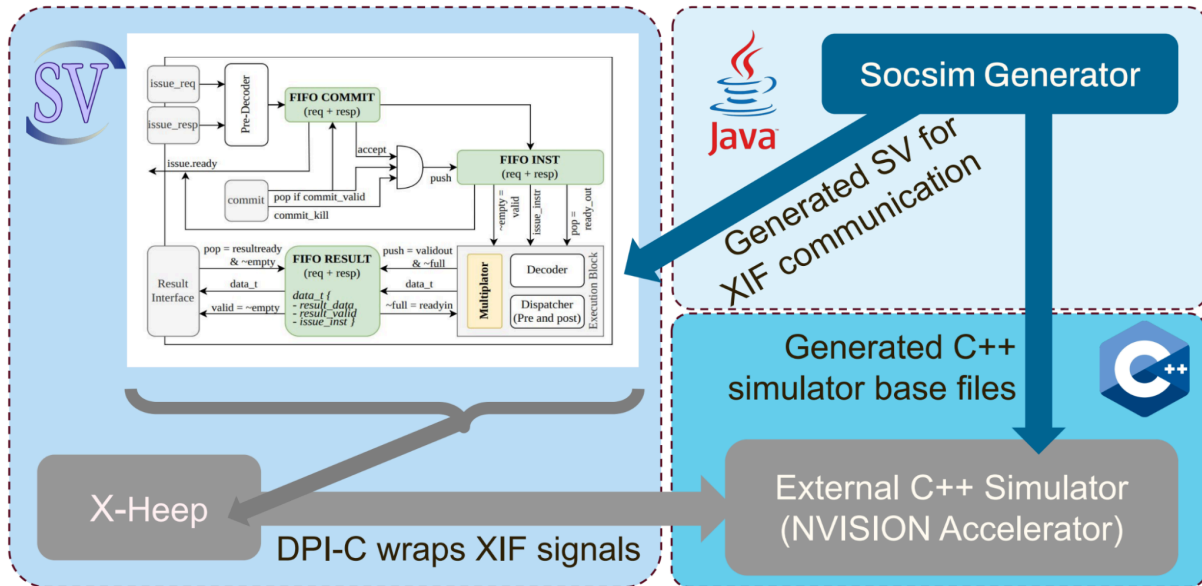


Figure 1: Simulator integration with Socsim Generator

The JSON configuration file specifies project metadata (e.g., project name and output paths), the type of the interface (OBI or XIF), and the target language for the simulator (C++, Java or Python). It also defines hardware-specific details, such as memory base addresses, register maps, and the direction and width of each signal. Custom instructions can be configured with their opcode, function fields and operand mapping. Finally, communication parameters need to be specified, including host and simulator addresses, timeouts and retry counts, to enable the generated simulator to interact with X-Heep. An example configuration for an XIF accelerator is illustrated in Figure 2.

```
{
  "project_name": "test_xif_accelerator",
  "paths": {
    "xheap_dir": "/home/usr/x-heap",
    "output_dir": "/home/usr/socsim-generator/examples"
  },
  "simulator_definitions": [
    {
      "interface": "xif",
      "identifier": "test_xif_accelerator",
      "description": "An XIF accelerator for testing",
      "class_name": "test_xif_accelerator",
      "target_language": "cpp",
      "hardware_interface": {
        "memory_base_address": 6000,
        "memory_size_bytes": 100,
        "registers": [
          { "name": "rs1_val", "direction_to_simulator": "input", "type": "logic", "signed": false, "width": 32, "default_value": 0, "description": "" },
          { "name": "rs2_val", "direction_to_simulator": "input", "type": "logic", "signed": false, "width": 32, "default_value": 0, "description": "" },
          { "name": "instr", "direction_to_simulator": "input", "type": "logic", "signed": false, "width": 32, "default_value": 0, "description": "" },
          { "name": "result_val", "direction_to_simulator": "output", "type": "logic", "signed": false, "width": 32, "default_value": 0, "description": "" }
        ]
      },
      "custom_instructions": [
        {
          "name": "test_instruction",
          "type": "r",
          "encoding": { "opcode": "0b0001011", "funct3": "0b000", "funct7": "0b00000000" },
          "operands": [ "rd", "rs1", "rs2", "instr" ],
          "operand_mapping": { "rs1": "rs1_val", "rs2": "rs2_val", "rd": "result_val", "instr": "instr" },
          "cycles": 1,
          "memory_access": false
        }
      ]
    }
  ],
  "communication": {
    "host_address": "tcp://localhost:5556",
    "simulator_address": "tcp://*:5556",
    "send_timeout_ms": 5000,
    "recv_timeout_ms": 5000,
    "retries": 3
  }
}
```

Figure 2: Example JSON configuration for Socsim Generator

3.2 NVISION Accelerator Co-Simulation

The Socsim Generator framework was used to co-simulate NVISION's MAC accelerator. The base code for the accelerator was first generated from a custom configured JSON file. On top of this generated high-level base code, the accelerator's behavior was implemented in C++. During co-simulation, XIF instructions from the processor core are intercepted, processed by the high-level simulator, and results are returned to the core, without requiring a manual hardware implementation.

The provided C++ snippet (Figure 2) illustrates the instruction dispatch mechanism used for the MAC accelerator. Instructions are decoded using the *funct3* field, which determines the specific operation to execute. This dispatch logic acts as a lightweight "instruction decoder" and routes incoming XIF instructions to the appropriate function in the simulator. Operands are read from the simulated registers, processed by the accelerator logic, and the results are written back to the destination registers, mimicking the behavior of a hardware implementation.

```
void busy (const socsim::simulation_data &in, socsim::simulation_data &out) override {
    uint32_t instr = this->get_instr();
    uint8_t funct3 = static_cast<uint8_t>((instr >> 12) & 0x7);
    switch (funct3) {
        case 0x2: { // MAC_8B
            this->parts_received++;
            if (this->parts_received >= 2) {
                this->parts_received = 0;
                __mac_8b(a:this->get_internal_rs1(), b:this->get_internal_rs2(), c:this->get_rs1_val(), d:this->get_rs2_val());
            } else {
                this->set_internal_rs1(this->get_rs1_val());
                this->set_internal_rs2(this->get_rs2_val());
            }
            break;
        }
    }
}
```

Figure 3: Snippet of NVISION MAC instructions dispatch logic

4. Measured Results

To evaluate the proposed implementation, NVISION's MAC benchmark program was used and executed on 784×64 matrices. Four implementations were considered: a baseline CPU version, an unrolled CPU variant, and two custom 8-bit implementations leveraging the external simulator through the XIF interface. Across all compiler optimization levels, the custom 8-bit implementations consistently achieved the highest speedups, with the unrolled version reaching up to a 13x performance improvement at the -O1 level. Speedups are computed relative to the CPU baseline compiled with the same optimization level.

Implementation	Cycles (-O0)	Speedup (-O0)	Cycles (-O1)	Speedup (-O1)	Cycles (-O2)	Speedup (-O2)
CPU	502234	1.0	552424	1.0	502234	1.0
CPU (unrolled)	238873	2.1	282791	1.95	238873	2.1
Custom 8-bit	69721	7.2	69678	7.93	69721	7.2
Custom 8-bit (unrolled)	41885	11.99	41972	13.16	41885	11.99

Table 1: Results obtained from NVISION MAC benchmark